Research Report: Geometric and Lie-Style Linguistic Transformations

Date

2026-06-11

Purpose

Fix today's research state for external verification. The report summarizes yesterday's geometric vector baseline and today's transition to composition, commutators, semantic controls, and Jacobi-like diagnostics.

Yesterday's fixed result

The original project supports the claim that linguistic transformations are recoverable as embedding displacement vectors. Delta representations remain useful across models and holdout regimes, but this does not by itself establish algebraic operator structure.

Today's fixed result

Composition order matters in embedding space. Semantic equivalence controls suggest the signal is not just wording noise. The useful algebraic diagnostic is not tautological antisymmetry, but third-order alternating cancellation compared against permutation-null baselines.

Main conclusion

The evidence supports local Lie-style behavior, not a global Lie algebra. The triple QMT shows robust cancellation across all tested models, while NQT and NMT are mixed or worse than null in several cases.

Jacobi-like summary with bootstrap CI

model	triple	mean_relative_jacobi_norm	relative_jacobi_norm_ci95_	relative_jacobi_norm_ci95_	mean_jacobi_to_null_mean	jacobi_to_null_mean_ratio_ci9	jacobi_to_null_mean_ratio_ci9	mean_null_percentile_smaller_is	n
bert-base-uncased	NMT	0.3048	0.3005	0.3090	0.7750	0.7669	0.7829	0.3522	100.0000
bert-base-uncased	NQM	0.2695	0.2660	0.2727	0.8628	0.8533	0.8720	0.3850	100.0000
bert-base-uncased	NQT	0.3719	0.3674	0.3763	1.1170	1.1082	1.1259	0.7005	100.0000
bert-base-uncased	QMT	0.2760	0.2712	0.2807	0.6833	0.6773	0.6892	0.1018	100.0000
distilroberta-base	NMT	0.3522	0.3485	0.3558	1.1338	1.1276	1.1404	0.7066	100.0000
distilroberta-base	NQM	0.2847	0.2809	0.2882	1.0821	1.0735	1.0914	0.7777	100.0000
distilroberta-base	NQT	0.2777	0.2752	0.2802	1.0803	1.0765	1.0841	0.6274	100.0000
distilroberta-base	QMT	0.1849	0.1832	0.1866	0.6311	0.6267	0.6356	0.0997	100.0000
roberta-base	NMT	0.4315	0.4255	0.4371	1.2532	1.2423	1.2637	0.8057	100.0000
roberta-base	NQM	0.3122	0.3086	0.3158	0.9799	0.9743	0.9853	0.6475	100.0000
roberta-base	NQT	0.3452	0.3413	0.3490	1.0805	1.0739	1.0871	0.7059	100.0000
roberta-base	QMT	0.2190	0.2160	0.2222	0.6229	0.6168	0.6287	0.1013	100.0000

Antisymmetry sanity check

model	pair	mean_comm_norm	mean_relative_antisym_error	mean_cos_comm_ab_neg_comm_ba	n
bert-base-uncased	MT_vs_TM	4.2108	0.0000	1.0000	100.0000
bert-base-uncased	NM_vs_MN	2.0965	0.0000	1.0000	100.0000
bert-base-uncased	NQ_vs_QN	4.0297	0.0000	1.0000	100.0000
bert-base-uncased	NT_vs_TN	4.9744	0.0000	1.0000	100.0000
bert-base-uncased	QM_vs_MQ	4.4676	0.0000	1.0000	100.0000
bert-base-uncased	QT_vs_TQ	3.0670	0.0000	1.0000	100.0000
distilroberta-base	MT_vs_TM	1.3342	0.0000	1.0000	100.0000
distilroberta-base	NM_vs_MN	1.2204	0.0000	1.0000	100.0000
distilroberta-base	NQ_vs_QN	1.4504	0.0000	1.0000	100.0000
distilroberta-base	NT_vs_TN	1.3915	0.0000	1.0000	100.0000
distilroberta-base	QM_vs_MQ	1.3567	0.0000	1.0000	100.0000
distilroberta-base	QT_vs_TQ	0.6474	0.0000	1.0000	100.0000
roberta-base	MT_vs_TM	1.3729	0.0000	1.0000	100.0000
roberta-base	NM_vs_MN	1.3408	0.0000	1.0000	100.0000
roberta-base	NQ_vs_QN	1.6280	0.0000	1.0000	100.0000
roberta-base	NT_vs_TN	1.4048	0.0000	1.0000	100.0000
roberta-base	QM_vs_MQ	1.5514	0.0000	1.0000	100.0000
roberta-base	QT_vs_TQ	0.7724	0.0000	1.0000	100.0000

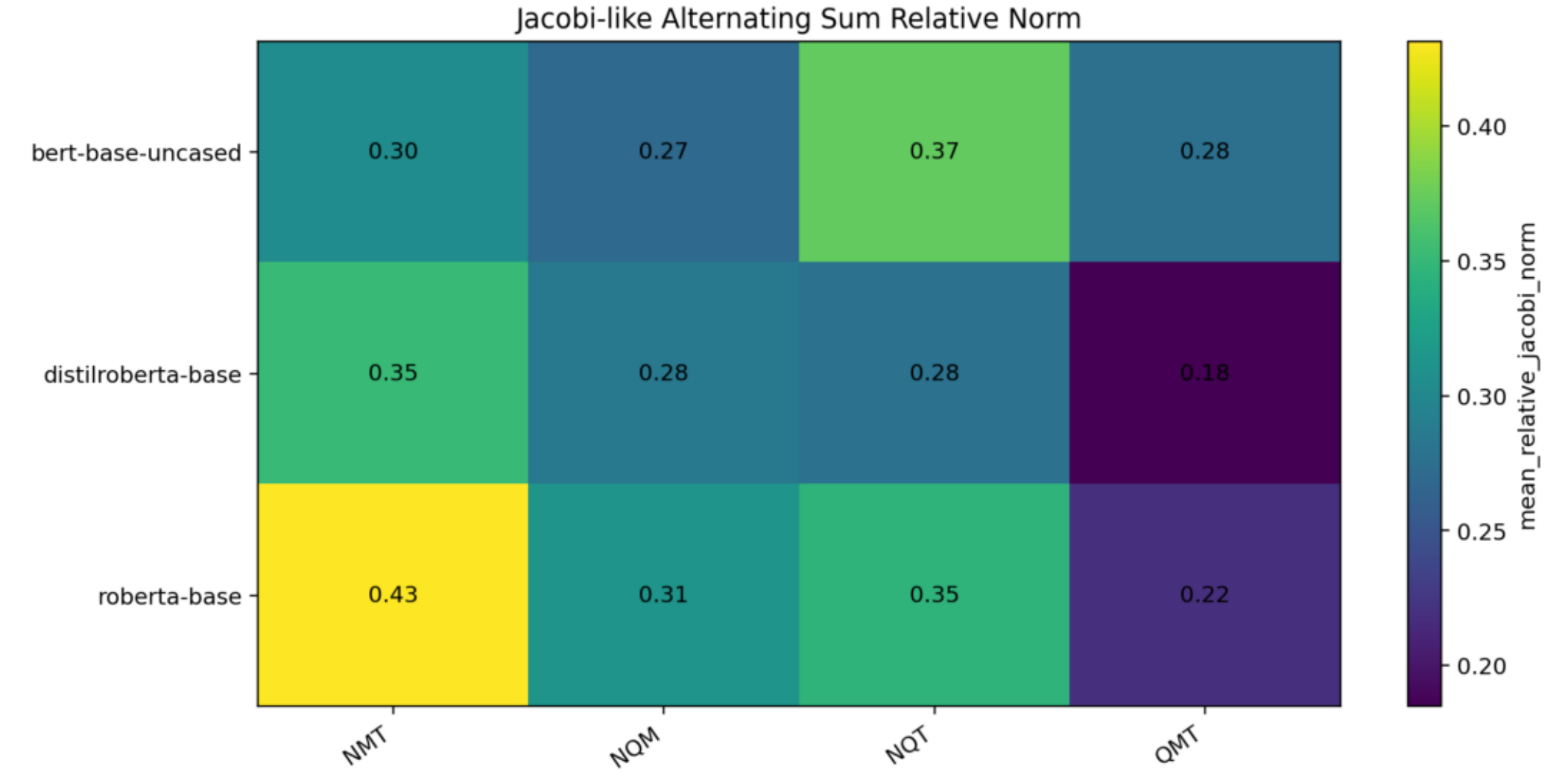
Semantic equivalence control

model	label	mean_noncommutativity	std_noncommutativity	mean_relative_commutator_std	std_relative_commutator_nor	mean_cosine	std_cosine	n
bert-base-uncased	equivalent	0.1130	0.0409	0.4818	0.0809	0.8870	0.0409	400.0000
bert-base-uncased	non_equivalent	0.3291	0.1134	0.8065	0.1326	0.6709	0.1134	400.0000
distilroberta-base	equivalent	0.1295	0.0483	0.5406	0.1277	0.8705	0.0483	400.0000
distilroberta-base	non_equivalent	0.2319	0.0381	0.6868	0.0560	0.7681	0.0381	400.0000
roberta-base	equivalent	0.1543	0.0630	0.5831	0.1304	0.8457	0.0630	400.0000
roberta-base	non_equivalent	0.2784	0.0439	0.7465	0.0597	0.7216	0.0439	400.0000

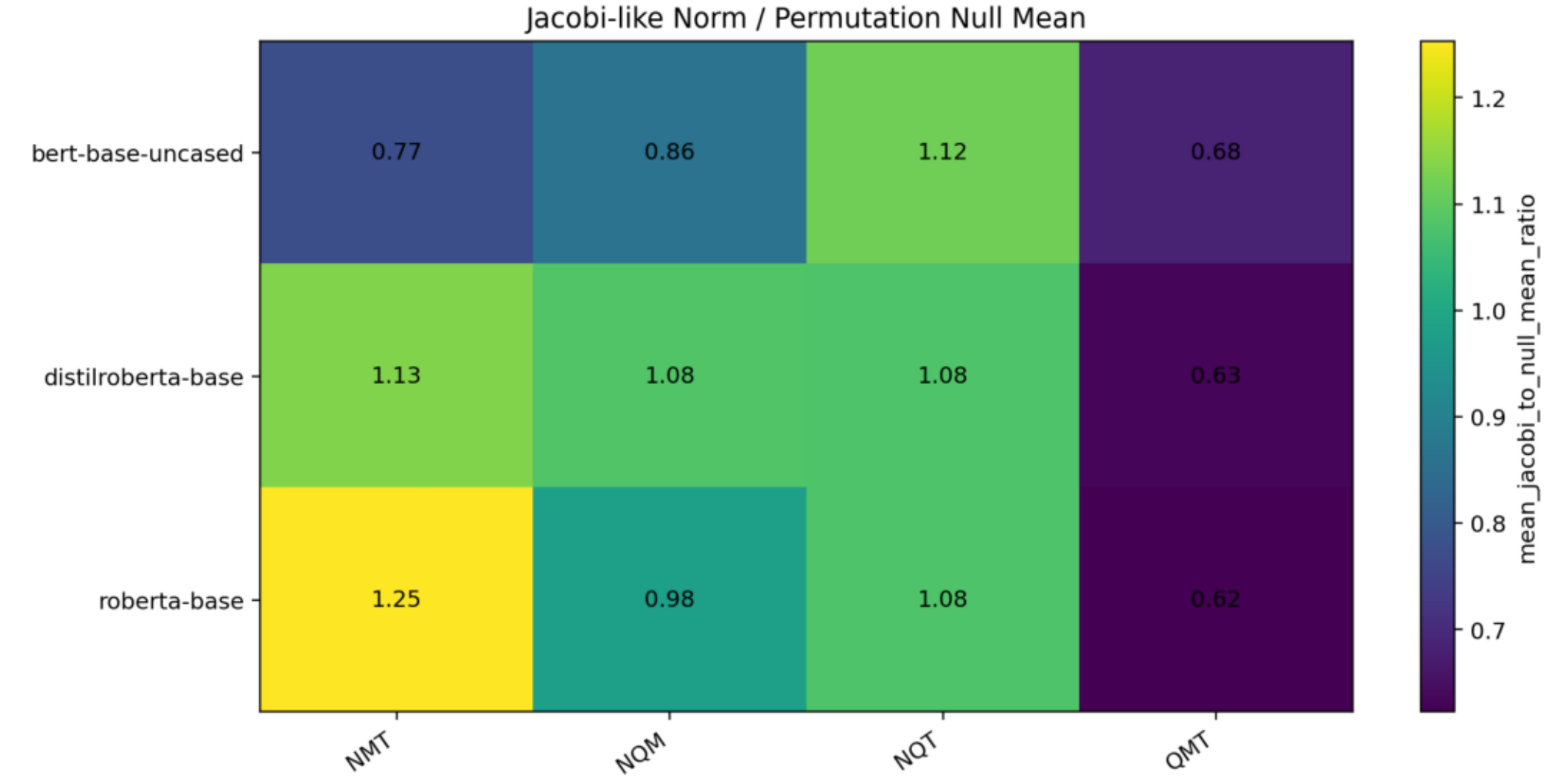
Composition summary: strongest noncommutativity rows

model	pair	mean_commutator_nor	std_commutator_norm	mean_relative_commutato	std_relative_commutator_r	mean_cosine	std_cosine	mean_noncommutativit	std_noncommutativity	n
bert-base-uncased	NT_vs_TN	5.1565	0.5832	0.9908	0.1066	0.5102	0.1009	0.4898	0.1009	80.0000
roberta-base	NQ_vs_QN	1.6422	0.0726	0.7846	0.0364	0.6928	0.0279	0.3072	0.0279	80.0000
bert-base-uncased	MT_vs_TM	4.2726	0.3075	0.7817	0.0954	0.6938	0.0727	0.3062	0.0727	80.0000
roberta-base	QM_vs_MQ	1.5649	0.0863	0.7714	0.0541	0.7032	0.0410	0.2968	0.0410	80.0000
distilroberta-base	MT_vs_TM	1.3504	0.0606	0.7683	0.0387	0.7124	0.0302	0.2876	0.0302	80.0000
bert-base-uncased	QM_vs_MQ	4.4793	0.2156	0.7341	0.0487	0.7340	0.0347	0.2660	0.0347	80.0000
roberta-base	MT_vs_TM	1.3956	0.0770	0.7215	0.0560	0.7411	0.0394	0.2589	0.0394	80.0000
roberta-base	NT_vs_TN	1.4185	0.1071	0.7172	0.0599	0.7429	0.0428	0.2571	0.0428	80.0000
bert-base-uncased	NQ_vs_QN	4.0903	0.2939	0.6970	0.0493	0.7598	0.0347	0.2402	0.0347	80.0000
distilroberta-base	NQ_vs_QN	1.4541	0.0644	0.6731	0.0384	0.7735	0.0257	0.2265	0.0257	80.0000
distilroberta-base	NT_vs_TN	1.3975	0.0725	0.6675	0.0342	0.7783	0.0232	0.2217	0.0232	80.0000
distilroberta-base	QM_vs_MQ	1.3672	0.0842	0.6613	0.0460	0.7926	0.0287	0.2074	0.0287	80.0000
bert-base-uncased	QT_vs_TQ	3.1001	0.1729	0.5463	0.0440	0.8513	0.0231	0.1487	0.0231	80.0000
roberta-base	QT_vs_TQ	0.7784	0.0701	0.3860	0.0342	0.9269	0.0128	0.0731	0.0128	80.0000
distilroberta-base	QT_vs_TQ	0.6580	0.0281	0.3422	0.0173	0.9415	0.0059	0.0585	0.0059	80.0000
bert-base-uncased	NM_vs_MN	1.2527	0.2368	0.2305	0.0538	0.9728	0.0125	0.0272	0.0125	80.0000
roberta-base	NM_vs_MN	0.3865	0.0403	0.2032	0.0248	0.9794	0.0050	0.0206	0.0050	80.0000
distilroberta-base	NM_vs_MN	0.3128	0.0212	0.1721	0.0201	0.9852	0.0036	0.0148	0.0036	80.0000

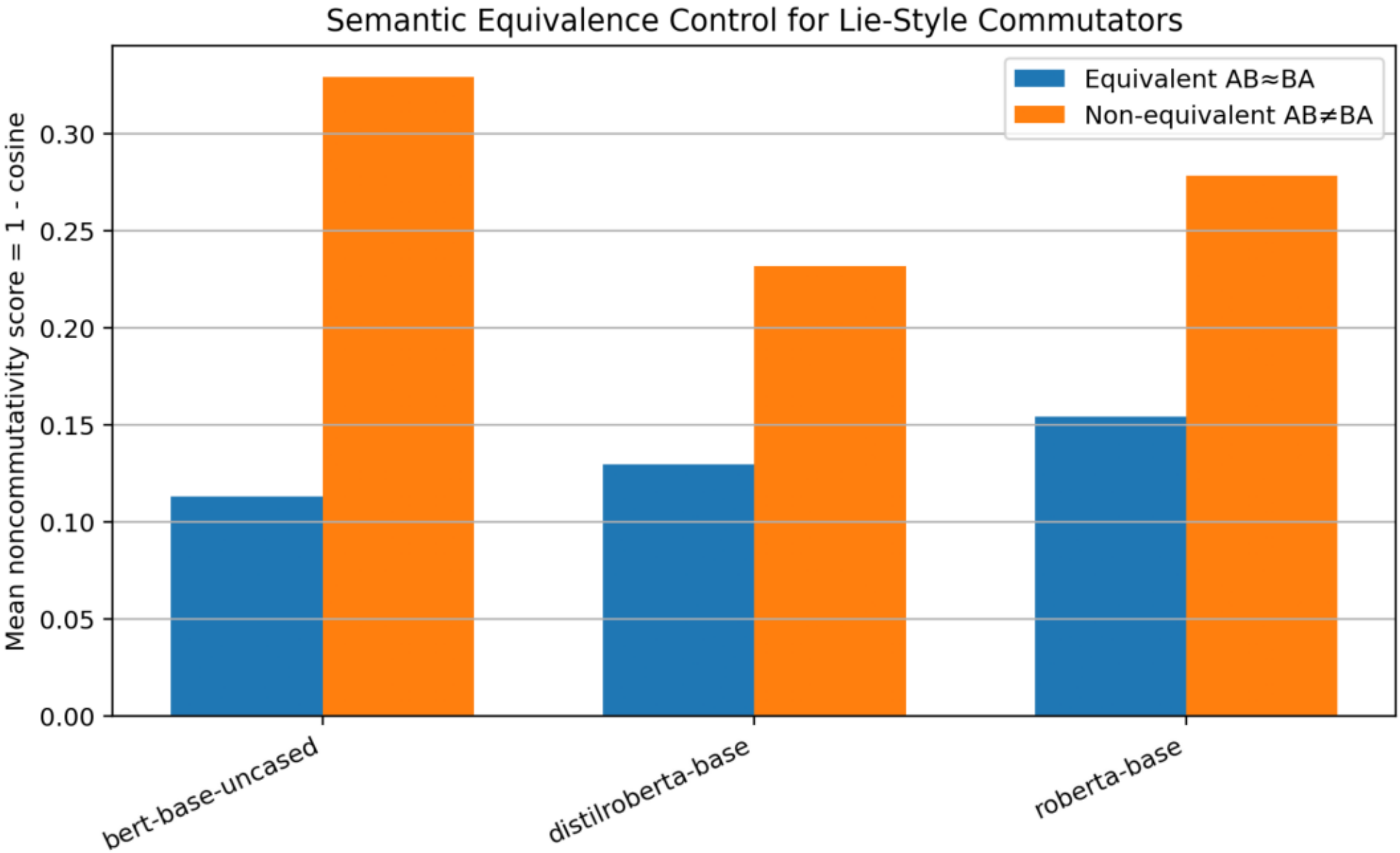
Jacobi-like relative norm



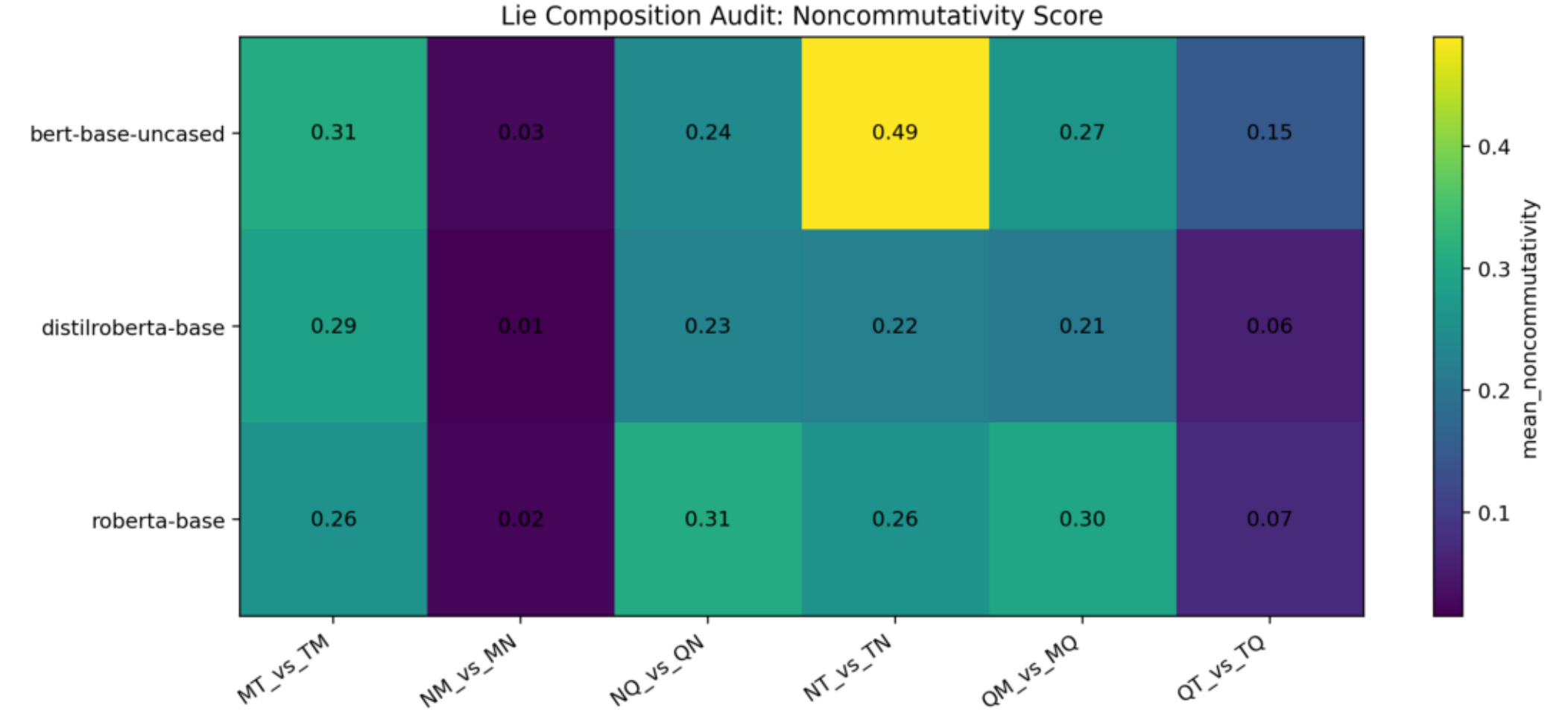
Jacobi-like norm versus permutation-null



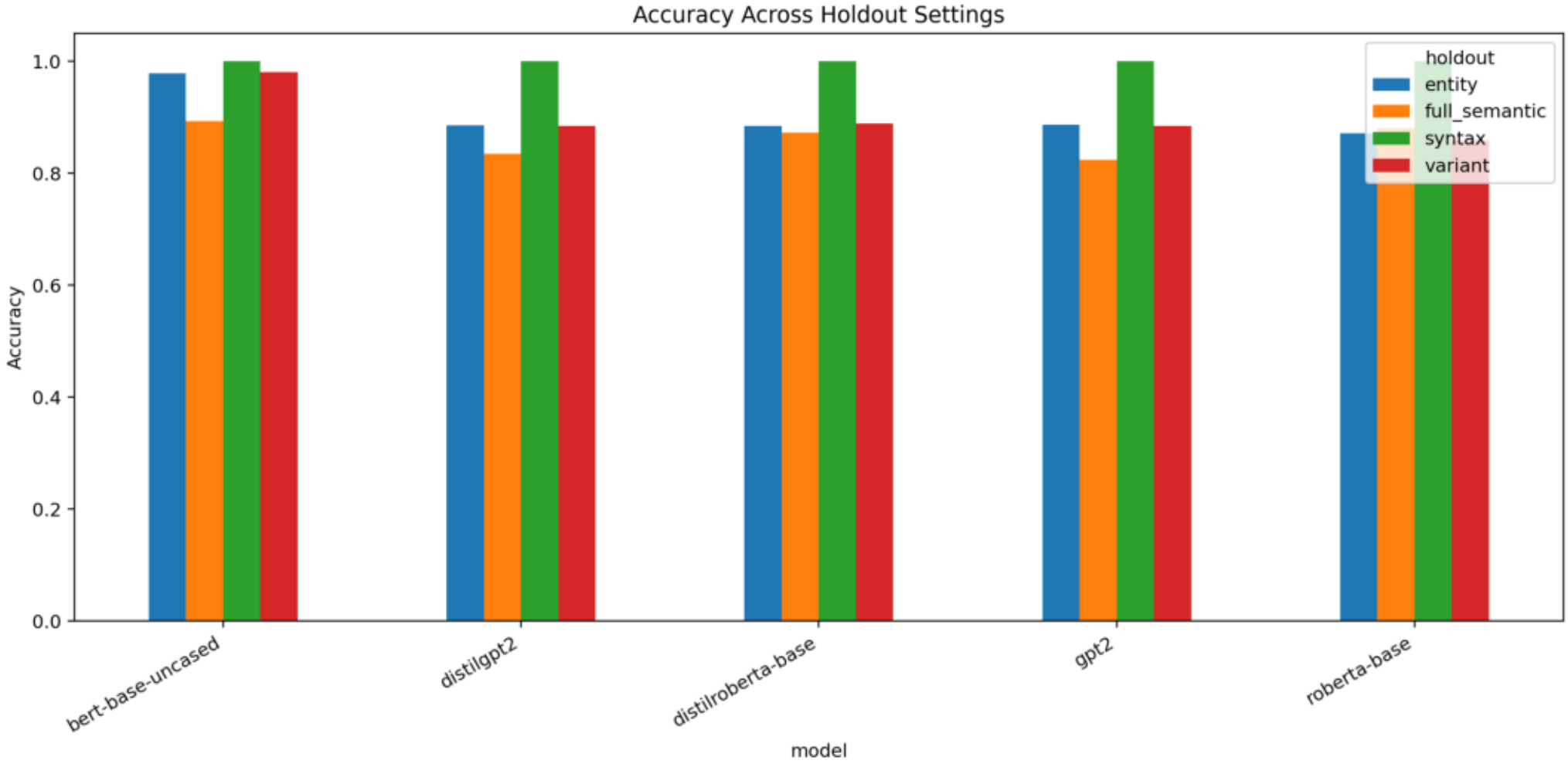
Semantic equivalent vs non-equivalent control



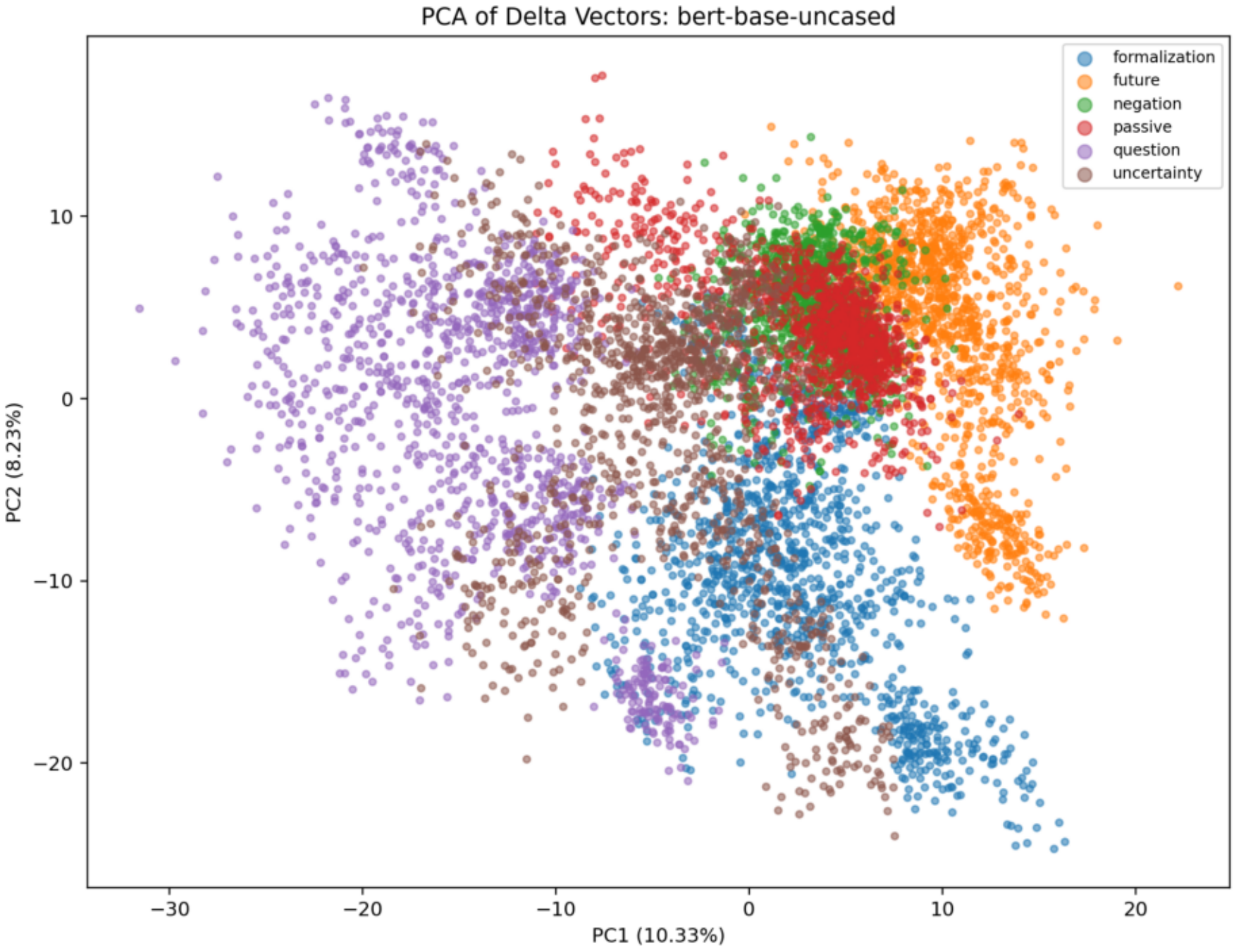
Composition noncommutativity heatmap



Original holdout accuracy comparison



Original PCA class geometry



Audit notes for external verifier

Antisymmetry is not treated as evidence because the implementation defines [B,A] as the negative order of [A,B].

A literal nested-commutator Jacobi expression over the same six endpoint vectors cancels algebraically. The reported diagnostic is the non-tautological alternating composition sum ABC+BCA+CAB-ACB-CBA-BAC.

Duplicate endpoint templates were detected and removed before finalizing today's result. The final dataset has 400 Jacobi rows and zero duplicate endpoint sets.

Regenerable .npy caches and one large all-pairs intermediate CSV were deleted after preserving summaries, raw CSVs, figures, and scripts.

Code listing: run_lie_algebraic_identities.py (1)

```
0001: import gc
0002: import json
0003: import warnings
0004: from pathlib import Path
0005:
0006: import numpy as np
0007: import pandas as pd
0008: import torch
0009: import matplotlib.pyplot as plt
0010:
0011: from transformers import AutoTokenizer, AutoModel
0012: from sklearn.decomposition import PCA
0013: from sklearn.metrics.pairwise import cosine_similarity
0014:
0015: warnings.filterwarnings("ignore")
0016:
0017:
0018: class LieAlgebraicIdentitiesAudit:
0019:     def __init__(self):
0020:         self.out_dir = Path("lie_algebraic_identities_results")
0021:         self.csv_dir = self.out_dir / "csv"
0022:         self.fig_dir = self.out_dir / "figures"
0023:         self.csv_dir.mkdir(parents=True, exist_ok=True)
0024:         self.fig_dir.mkdir(parents=True, exist_ok=True)
0025:
0026:         self.device = "cuda" if torch.cuda.is_available() else "cpu"
0027:         self.hf_token = None
0028:         self.pca_dim = 64
0029:         self.n_jacobi_null = 1000
0030:         self.n_bootstrap = 2000
0031:
0032:         self.models = [
0033:             "bert-base-uncased",
0034:             "distilroberta-base",
0035:             "roberta-base",
0036:         ]
0037:
0038:         self.ops = ["N", "Q", "M", "T"]
0039:
0040:         self.subjects = [
0041:             "scientist", "engineer", "teacher", "doctor", "programmer",
0042:             "researcher", "analyst", "manager", "wizard", "dragon",
0043:             "queen", "robot", "pirate", "oracle", "knight", "alien",
0044:         ]
0045:
0046:         self.actions = [
0047:             ("accepted", "accept", "the explanation"),
0048:             ("completed", "complete", "the repair"),
0049:             ("confirmed", "confirm", "the answer"),
0050:             ("approved", "approve", "the treatment"),
0051:             ("fixed", "fix", "the bug"),
0052:             ("supported", "support", "the theory"),
0053:             ("verified", "verify", "the report"),
0054:             ("guarded", "guard", "the treasure"),
```

Code listing: run_lie_algebraic_identities.py (2)

```
            ("opened", "open", "the portal"),
            ("signed", "sign", "the treaty"),
        ]

    def log(self, msg):
        print(msg, flush=True)

    def base_sentence(self, subject, past, obj):
        return f"The {subject} {past} {obj}."

    def compose(self, seq, subject, past, base, obj):
        s = "".join(seq)

        mapping = {
            "": f"The {subject} {past} {obj}.",

            "N": f"The {subject} failed to {base} {obj}.",
            "Q": f"Could the {subject} {base} {obj}?",
            "M": f"The {subject} allegedly {past} {obj}.",
            "T": f"The {subject} will {base} {obj} tomorrow.",

            "NQ": f"Could the {subject} fail to {base} {obj}?",
            "QN": f"Is it false that the {subject} {past} {obj}?",

            "NM": f"The {subject} allegedly failed to {base} {obj}.",
            "MN": f"Allegedly, the {subject} failed to {base} {obj}.",

            "NT": f"The {subject} will fail to {base} {obj} tomorrow.",
            "TN": f"The {subject} failed to have {past} {obj} earlier.",

            "QM": f"Could the {subject} allegedly {base} {obj}?",
            "MQ": f"Is it alleged that the {subject} {past} {obj}?",

            "QT": f"Could the {subject} {base} {obj} tomorrow?",
            "TQ": f"Will the {subject} {base} {obj} tomorrow?",

            "MT": f"The {subject} will allegedly {base} {obj} tomorrow.",
            "TM": f"The {subject} was allegedly going to {base} {obj}.",
        }

        if s in mapping:
            return mapping[s]

        # canonical triple templates
        if s == "NQM":
            return f"Could the {subject} allegedly fail to {base} {obj}?"
        if s == "QMN":
            return f"Is it false that the {subject} allegedly {past} {obj}?"
        if s == "MNQ":
            return f"Is it alleged that the {subject} failed to {base} {obj}?"

        if s == "QNM":
            return f"Could it be false that the {subject} allegedly {past} {obj}?"
        if s == "NMQ":
```

Code listing: run_lie_algebraic_identities.py (3)

```
            return f"Could it be alleged that the {subject} failed to {base} {obj}?"
        if s == "MQN":
            return f"Allegedly, is it false that the {subject} {past} {obj}?"

        if s == "NMT":
            return f"The {subject} will allegedly fail to {base} {obj} tomorrow."
        if s == "MTN":
            return f"The {subject} was allegedly going to fail to {base} {obj}."
        if s == "TNM":
            return f"Earlier, the {subject} allegedly failed to have {past} {obj}."

        if s == "NTM":
            return f"Allegedly, the {subject} will fail to {base} {obj} tomorrow."
        if s == "TMN":
            return f"Earlier, it was alleged that the {subject} failed to have {past} {obj}."
        if s == "MNT":
            return f"It is alleged that the {subject} will fail to {base} {obj} tomorrow."

        if s == "NQT":
            return f"Could the {subject} fail to {base} {obj} tomorrow?"
        if s == "QTN":
            return f"Is it false that the {subject} will {base} {obj} tomorrow?"
        if s == "TNQ":
            return f"Could the {subject} have failed to {base} {obj} earlier?"

        if s == "QNT":
            return f"Could it be false that the {subject} will {base} {obj} tomorrow?"
        if s == "NTQ":
            return f"Could it be that the {subject} will fail to {base} {obj} tomorrow?"
        if s == "TQN":
            return f"Will it be false that the {subject} {base} {obj} tomorrow?"

        if s == "QMT":
            return f"Could the {subject} allegedly {base} {obj} tomorrow?"
        if s == "MTQ":
            return f"Could it be alleged that the {subject} will {base} {obj} tomorrow?"
        if s == "TQM":
            return f"Allegedly, will the {subject} {base} {obj} tomorrow?"

        if s == "MQT":
            return f"Will it be alleged that the {subject} {base} {obj} tomorrow?"
        if s == "QTM":
            return f"Allegedly, could the {subject} {base} {obj} tomorrow?"
        if s == "TMQ":
            return f"Could the {subject} have allegedly planned to {base} {obj}?"

        raise ValueError(f"Unsupported composition: {s}")

    def build_dataset(self, n_templates=100, seed=42):
        rng = np.random.default_rng(seed)
        combos = [(s, a) for s in self.subjects for a in self.actions]
        rng.shuffle(combos)
        combos = combos[:n_templates]

```

Code listing: run_lie_algebraic_identities.py (4)

```
        rows = []

        pairs = [("N", "Q"), ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
        triples = [("N", "Q", "M"), ("N", "Q", "T"), ("N", "M", "T"), ("Q", "M", "T")]

        for i, (subject, action) in enumerate(combos):
            past, base, obj = action
            source = self.base_sentence(subject, past, obj)

            for a, b in pairs:
                rows.append({
                    "template_id": i,
                    "kind": "antisymmetry",
                    "source": source,
                    "a": a,
                    "b": b,
                    "c": "",
                    "ab": self.compose([a, b], subject, past, base, obj),
                    "ba": self.compose([b, a], subject, past, base, obj),
                })

            for a, b, c in triples:
                rows.append({
                    "template_id": i,
                    "kind": "jacobi",
                    "source": source,
                    "a": a,
                    "b": b,
                    "c": c,
                    "abc": self.compose([a, b, c], subject, past, base, obj),
                    "bca": self.compose([b, c, a], subject, past, base, obj),
                    "cab": self.compose([c, a, b], subject, past, base, obj),
                    "acb": self.compose([a, c, b], subject, past, base, obj),
                    "cba": self.compose([c, b, a], subject, past, base, obj),
                    "bac": self.compose([b, a, c], subject, past, base, obj),
                })

        df = pd.DataFrame(rows)
        df.to_csv(self.csv_dir / "lie_algebraic_identities_dataset.csv", index=False)
        return df

    @torch.no_grad()
    def get_embeddings(self, model_name, texts, batch_size=16):
        tokenizer = AutoTokenizer.from_pretrained(model_name, token=self.hf_token)

        if tokenizer.pad_token is None:
            tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"

        model = AutoModel.from_pretrained(model_name, token=self.hf_token).to(self.device)
        model.eval()

        embs = []

        for i in range(0, len(texts), batch_size):
```

Code listing: run_lie_algebraic_identities.py (5)

```
            batch = texts[i:i + batch_size]

            enc = tokenizer(
                batch,
                padding=True,
                truncation=True,
                max_length=128,
                return_tensors="pt",
            ).to(self.device)

            out = model(**enc)
            hidden = out.last_hidden_state
            mask = enc["attention_mask"].unsqueeze(-1)

            pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
            embs.append(pooled.detach().cpu().numpy())

        del model
        gc.collect()

        if self.device == "cuda":
            torch.cuda.empty_cache()

        return np.vstack(embs)

    def cos(self, x, y):
        return float(cosine_similarity(x.reshape(1, -1), y.reshape(1, -1))[0, 0])

    def alternating_null_stats(self, vectors, observed_norm, scale, rng):
        null_relative = []
        n = len(vectors)

        for _ in range(self.n_jacobi_null):
            positive = set(rng.choice(n, size=n // 2, replace=False).tolist())
            signed_sum = np.zeros_like(vectors[0])

            for i, vector in enumerate(vectors):
                signed_sum += vector if i in positive else -vector

            null_relative.append(np.linalg.norm(signed_sum) / scale)

        null_relative = np.asarray(null_relative, dtype=float)
        observed_relative = observed_norm / scale

        return {
            "null_mean_relative_jacobi_norm": float(null_relative.mean()),
            "null_std_relative_jacobi_norm": float(null_relative.std(ddof=1)),
            "jacobi_to_null_mean_ratio": float(observed_relative / (null_relative.mean() + 1e-12)),
            "null_percentile_smaller_is_better": float((null_relative <= observed_relative).mean()),
        }

    def bootstrap_mean_ci(self, values, rng, alpha=0.05):
        values = np.asarray(values, dtype=float)
        if len(values) == 0:
```

Code listing: run_lie_algebraic_identities.py (6)

```
            return np.nan, np.nan

        draws = rng.choice(values, size=(self.n_bootstrap, len(values)), replace=True).mean(axis=1)
        return (
            float(np.quantile(draws, alpha / 2)),
            float(np.quantile(draws, 1 - alpha / 2)),
        )

    def compute_for_model(self, model_name, df):
        self.log("\n" + "=" * 80)
        self.log(f"MODEL: {model_name}")
        self.log("=" * 80)

        text_cols = ["source", "ab", "ba", "abc", "bca", "cab", "acb", "cba", "bac"]
        all_texts = set()

        for col in text_cols:
            if col in df.columns:
                all_texts |= set(df[col].dropna().values)

        all_texts = sorted(all_texts)
        idx = {t: i for i, t in enumerate(all_texts)}

        raw = self.get_embeddings(model_name, all_texts)

        dim = min(self.pca_dim, raw.shape[0] - 1, raw.shape[1])
        pca = PCA(n_components=dim, random_state=42)
        vecs = pca.fit_transform(raw)

        anti_rows = []
        jacobi_rows = []

        anti_df = df[df["kind"] == "antisymmetry"]

        for row in anti_df.itertuples():
            x = vecs[idx[row.source]]
            ab = vecs[idx[row.ab]]
            ba = vecs[idx[row.ba]]

            delta_ab = ab - x
            delta_ba = ba - x

            comm_ab = delta_ab - delta_ba
            comm_ba = delta_ba - delta_ab

            antisym_error = np.linalg.norm(comm_ab + comm_ba)
            comm_norm = np.linalg.norm(comm_ab)

            anti_rows.append({
                "model": model_name,
                "template_id": row.template_id,
                "pair": f"{row.a}{row.b}_vs_{row.b}{row.a}",
                "a": row.a,
                "b": row.b,
```

Code listing: run_lie_algebraic_identities.py (7)

```
                "comm_norm": float(comm_norm),
                "antisym_error": float(antisym_error),
                "relative_antisym_error": float(antisym_error / (comm_norm + 1e-12)),
                "cos_comm_ab_neg_comm_ba": self.cos(comm_ab, -comm_ba),
            })

        jacobi_df = df[df["kind"] == "jacobi"]
        seed = sum((i + 1) * ord(ch) for i, ch in enumerate(model_name)) % (2 ** 32)
        rng = np.random.default_rng(seed)

        for row in jacobi_df.itertuples():
            x = vecs[idx[row.source]]

            abc = vecs[idx[row.abc]] - x
            bca = vecs[idx[row.bca]] - x
            cab = vecs[idx[row.cab]] - x

            acb = vecs[idx[row.acb]] - x
            cba = vecs[idx[row.cba]] - x
            bac = vecs[idx[row.bac]] - x

            # Jacobi-like alternating sum over permutations:
            # J = ABC + BCA + CAB - ACB - CBA - BAC
            jacobi = abc + bca + cab - acb - cba - bac

            positive_norm = np.linalg.norm(abc) + np.linalg.norm(bca) + np.linalg.norm(cab)
            negative_norm = np.linalg.norm(acb) + np.linalg.norm(cba) + np.linalg.norm(bac)
            scale = 0.5 * (positive_norm + negative_norm) + 1e-12

            jacobi_norm = np.linalg.norm(jacobi)
            null_stats = self.alternating_null_stats(
                [abc, bca, cab, acb, cba, bac],
                jacobi_norm,
                scale,
                rng,
            )

            jacobi_rows.append({
                "model": model_name,
                "template_id": row.template_id,
                "triple": f"{row.a}{row.b}{row.c}",
                "a": row.a,
                "b": row.b,
                "c": row.c,
                "jacobi_norm": float(jacobi_norm),
                "relative_jacobi_norm": float(jacobi_norm / scale),
                **null_stats,
            })

        anti_result = pd.DataFrame(anti_rows)
        jacobi_result = pd.DataFrame(jacobi_rows)

        anti_result.to_csv(self.csv_dir / f"antisymmetry_raw_{self.safe_name(model_name)}.csv", index=False)
        jacobi_result.to_csv(self.csv_dir / f"jacobi_raw_{self.safe_name(model_name)}.csv", index=False)
```

Code listing: run_lie_algebraic_identities.py (8)

```
0379:
0380:         return anti_result, jacobi_result
0381:
0382:     def safe_name(self, model_name):
0383:         return model_name.replace("/", "_").replace("-", "_")
0384:
0385:     def summarize(self, anti_all, jacobi_all):
0386:         anti_summary = (
0387:             anti_all.groupby(["model", "pair"])
0388:             .agg(
0389:                 mean_comm_norm=("comm_norm", "mean"),
0390:                 mean_relative_antisym_error=("relative_antisym_error", "mean"),
0391:                 mean_cos_comm_ab_neg_comm_ba=("cos_comm_ab_neg_comm_ba", "mean"),
0392:                 n=("pair", "count"),
0393:             )
0394:             .reset_index()
0395:         )
0396:
0397:         jacobi_summary = (
0398:             jacobi_all.groupby(["model", "triple"])
0399:             .agg(
0400:                 mean_jacobi_norm=("jacobi_norm", "mean"),
0401:                 mean_relative_jacobi_norm=("relative_jacobi_norm", "mean"),
0402:                 mean_null_relative_jacobi_norm=("null_mean_relative_jacobi_norm", "mean"),
0403:                 mean_jacobi_to_null_mean_ratio=("jacobi_to_null_mean_ratio", "mean"),
0404:                 mean_null_percentile_smaller_is_better=("null_percentile_smaller_is_better", "mean"),
0405:                 n=("triple", "count"),
0406:             )
0407:             .reset_index()
0408:         )
0409:
0410:         rng = np.random.default_rng(20260611)
0411:         ci_rows = []
0412:
0413:         for row in jacobi_summary.itertuples():
0414:             group = jacobi_all[
0415:                 (jacobi_all["model"] == row.model)
0416:                 & (jacobi_all["triple"] == row.triple)
0417:             ]
0418:
0419:             rel_low, rel_high = self.bootstrap_mean_ci(group["relative_jacobi_norm"].values, rng)
0420:             ratio_low, ratio_high = self.bootstrap_mean_ci(group["jacobi_to_null_mean_ratio"].values, rng)
0421:
0422:             ci_rows.append({
0423:                 "model": row.model,
0424:                 "triple": row.triple,
0425:                 "relative_jacobi_norm_ci95_low": rel_low,
0426:                 "relative_jacobi_norm_ci95_high": rel_high,
0427:                 "jacobi_to_null_mean_ratio_ci95_low": ratio_low,
0428:                 "jacobi_to_null_mean_ratio_ci95_high": ratio_high,
0429:             })
0430:
0431:         jacobi_summary = jacobi_summary.merge(pd.DataFrame(ci_rows), on=["model", "triple"], how="left")
0432:
```

Code listing: run_lie_algebraic_identities.py (9)

```
        anti_summary.to_csv(self.csv_dir / "antisymmetry_summary.csv", index=False)
        jacobi_summary.to_csv(self.csv_dir / "jacobi_summary.csv", index=False)

        return anti_summary, jacobi_summary

    def plot_heatmap(self, df, index_col, value_col, filename, title):
        rows = sorted(df["model"].unique())
        cols = sorted(df[index_col].unique())

        mat = np.zeros((len(rows), len(cols)))

        for i, model in enumerate(rows):
            for j, col in enumerate(cols):
                val = df[(df["model"] == model) & (df[index_col] == col)][value_col].values
                mat[i, j] = val[0] if len(val) else np.nan

        plt.figure(figsize=(10, 5))
        plt.imshow(mat, aspect="auto")
        plt.colorbar(label=value_col)

        plt.xticks(np.arange(len(cols)), cols, rotation=35, ha="right")
        plt.yticks(np.arange(len(rows)), rows)
        plt.title(title)

        for i in range(mat.shape[0]):
            for j in range(mat.shape[1]):
                plt.text(j, i, f"{mat[i, j]:.2f}", ha="center", va="center")

        path = self.fig_dir / filename
        plt.tight_layout()
        plt.savefig(path, dpi=220, bbox_inches="tight")
        plt.close()
        self.log(f"Saved figure: {path}")

    def run(self):
        self.log(f"DEVICE: {self.device}")
        self.log(f"OUT DIR: {self.out_dir}")

        df = self.build_dataset(n_templates=100, seed=42)

        anti_results = []
        jacobi_results = []

        for model in self.models:
            anti, jacobi = self.compute_for_model(model, df)
            anti_results.append(anti)
            jacobi_results.append(jacobi)

        anti_all = pd.concat(anti_results, ignore_index=True)
        jacobi_all = pd.concat(jacobi_results, ignore_index=True)

        anti_all.to_csv(self.csv_dir / "antisymmetry_raw_all_models.csv", index=False)
        jacobi_all.to_csv(self.csv_dir / "jacobi_raw_all_models.csv", index=False)

```

Code listing: run_lie_algebraic_identities.py (10)

```
        anti_summary, jacobi_summary = self.summarize(anti_all, jacobi_all)

        self.plot_heatmap(
            anti_summary,
            "pair",
            "mean_cos_comm_ab_neg_comm_ba",
            "01_antisymmetry_cosine_heatmap.png",
            "Antisymmetry Check: cos([A,B], -[B,A])",
        )

        self.plot_heatmap(
            anti_summary,
            "pair",
            "mean_relative_antisym_error",
            "02_antisymmetry_error_heatmap.png",
            "Antisymmetry Error",
        )

        self.plot_heatmap(
            jacobi_summary,
            "triple",
            "mean_relative_jacobi_norm",
            "03_jacobi_relative_norm_heatmap.png",
            "Jacobi-like Alternating Sum Relative Norm",
        )

        self.plot_heatmap(
            jacobi_summary,
            "triple",
            "mean_jacobi_to_null_mean_ratio",
            "04_jacobi_vs_permutation_null_heatmap.png",
            "Jacobi-like Norm / Permutation Null Mean",
        )

        metadata = {
            "antisymmetry": "checks whether [A,B] ≈ -[B,A]",
            "jacobi_like": "checks alternating sum ABC+BCA+CAB-ACB-CBA-BAC",
            "jacobi_null": (
                "compares the observed alternating sum with random 3-positive/3-negative "
                "sign assignments over the same six composition vectors; lower ratio and "
                "lower percentile mean stronger Jacobi-like cancellation"
            ),
            "caution": (
                "A literal nested-commutator Jacobi expansion over the same six embedded "
                "composition endpoints cancels by construction, so this script reports a "
                "non-tautological alternating third-order cancellation diagnostic instead."
            ),
            "note": "This is an empirical Lie-style diagnostic, not a formal proof of Lie algebra.",
            "models": self.models,
            "n_jacobi_null": self.n_jacobi_null,
            "n_bootstrap": self.n_bootstrap,
            "jacobi_triples": ["NQM", "NQT", "NMT", "QMT"],
        }

```

Code listing: run_lie_algebraic_identities.py (11)

```
        with open(self.out_dir / "metadata.json", "w", encoding="utf-8") as f:
            json.dump(metadata, f, indent=2)

        self.log("\nANTISYMMETRY SUMMARY")
        self.log(str(anti_summary))

        self.log("\nJACOBI SUMMARY")
        self.log(str(jacobi_summary))

        self.log("\nDONE")


if __name__ == "__main__":
    audit = LieAlgebraicIdentitiesAudit()
    audit.run()
```